SAHAAYA

Every Need. Every Voice. Every Me.

M1: Approach

Project focus
A multilingual, accessible restaurant companion supporting the customer from menu discovery and ordering through staff communication, bill review and feedback.

1. Problem statement and understanding

Restaurant ordering often assumes that customers can read printed menus, hear staff, speak clearly and navigate conventional interfaces. These assumptions create barriers for people with visual, hearing or speech impairments, particularly when menu details, order changes and bills are available in only one format. SAHAAYA aims to make the full dining journey accessible through a multilingual interface that adapts how information is received and expressed. The Megathon brief requires visual impairment plus two other conditions, with a specific demonstrable feature for each.

Selected conditions and features

Condition	Barrier	Demonstrable feature
Visual impairment (mandatory)	Printed menus, item lists and bills may not be accessible visually.	Scan/OCR a physical menu; read items and prices aloud; provide voice-first navigation and spoken bill/table descriptions.
Hearing impairment	Spoken waiter responses and audio-only alerts may be missed.	Speech-to-text captions, visible order confirmations and visual alerts.
Speech impairment	Voice input may not capture the customer's request reliably.	Phrase board and typed requests with text-to-speech for waiter communication.

2. Idea or abstract

SAHAAYA is a Progressive Web App (PWA) that supports the restaurant journey: scan a physical menu, understand dishes and prices, filter or compare items, confirm an order, communicate with staff, review the bill, split the total and provide feedback. It combines OCR, speech recognition, machine translation and text-to-speech with an interaction engine. English, Hindi and Telugu are planned, with input and output languages selected independently. The app uses available accessibility signals and in-app interaction patterns to offer suitable features; users can confirm, change or undo a suggested mode. This is an inference of accessibility needs, not a diagnosis.

3. High-level approach, pipeline and architecture

Customer PWA	Backend/API proxy	Services and data
Camera/upload, microphone or typing, phrase board, accessible menu, cart, staff and bill screens.	HTTPS APIs, session state, menu parser, order and bill calculations, inference logs, retries and normalized errors.	AnuvaadHub/Bhashini OCR, ASR, translation and TTS services; sample menu JSON; translated templates and phrase audio.

- **Journey:** Start/unlock → scan or select sample menu → explore/filter/compare → cart and order confirmation → customer–staff communication with captions → accessible bill, split and confirmation → feedback.
- **Speech-to-speech:** ASR → translation to the AI stage's language → response generation → translation to the selected output language → TTS. The demo will show a live Bhashini/AnuvaadHub exchange where supported.
- **Menu pipeline:** check image blur/brightness → OCR → extract sections, items and prices with confidence → read results back → request retake or correction for unclear fields. Dietary/allergen information must be confirmed with restaurant staff.
- **Adaptation engine:** combine supported accessibility settings and in-app signals; score with decay, explain the inference, and offer one-tap confirmation and undo. Never infer a disability from silence alone.
- **Accessible output and resilience:** captions/transcripts, adjustable text and speech speed, explain-simpler, semantic headings/lists, and describe-in-words for menu/comparison/bill tables. Handle low-confidence ASR and network drops with retry, repeat and confirmation prompts.

Initial API contract

Endpoint	Input → output
POST /api/asr	Audio + language → text + confidence.
POST /api/mt	Text + source/target languages → translated text.
POST /api tts	Text + language → audio payload or URL.
POST /api/menu /parse	Image → sections, items, unclear fields and confidence.

4. Planned testing methodology

Test area	Method and expected evidence
Condition-specific tasks	Demonstrate visual impairment from menu to bill; hearing impairment using live captions and visual confirmations; and speech impairment using phrase-board/TTS communication. Each condition must have a distinct feature.
Language and Bhashini	Test English, Hindi and Telugu input/output with short restaurant prompts. Verify supported language pairs and record service response, latency and unsupported combinations
Menu/OCR	Test sample and photographed menus under clear, blurred and dim conditions. Check item/section extraction and price parsing; unclear fields must prompt a retake or confirmation rather than silent guessing
Speech and staff relay	Test clear, noisy and deliberately misrecognized speech; verify correction prompts, two-way customer/staff messages, captions and translation
Accessible outputs	Compare raw and accessible views. Check screen-reader labels, headings/lists, live announcements, adjustable text/speech speed, explain-simpler and describe-in-words for menu, comparison and bill tables.
Failure handling	Simulate low-confidence ASR, latency, timeouts and network loss. Verify retry/confirm flows and cached phrases; ensure orders and simulated payments are never reported successful without confirmation.

Testing evidence

For each run, record the task, condition-specific feature, language pair, expected and actual result, OCR/ASR confidence where available, response time, and whether the app recovered correctly from an error. The prototype will be considered ready for the demo when all three condition-specific tasks work, a live Bhashini speech-to-speech exchange is shown, raw and accessible outputs can be compared, and misrecognition/network failures are handled visibly.